

Genesis Engineering Specification (GES)

Project Genesis Research Repository

Generated: July 2026

Genesis Engineering Specification (GES)

Version 1.0 APIs, Schemas, Database & Deployment

\textbf{Project}: Project Genesis Agentic Artificial Life & Evolutionary Biology Simulator \textbf{Document Role}: Technical contracts, data schemas, infrastructure \textbf{Companion Documents}: GPS (Product) GDS (Design System) GCR (Content & Research) \textbf{Status}: Living document update with every API change or schema revision

> "The engineering specification is the contract between the simulation engine and the portal. If they disagree, the spec is the arbiter."

Part I Repository Structure

1.1 Why "portal" and not "website"

The directory is named \texttt{portal/}, not \texttt{website/}.

"Website" describes the implementation (HTML served over HTTP). "Portal" describes the purpose (a gateway to Genesis's scientific output).

Five years from now, the portal may include interactive simulations, community tools, downloadable datasets, and public APIs. The name must scale with the ambition.

1.2 Recommended Repository Layout

```
project-genesis/
    engine/                                # Renamed from world/
    the simulation core
        state.py
        biomes.py
        climate.py
        climate_epochs.py
        erosion.py
        evolution_journal.py
        explainer.py
        generator.py
        habitability.py
        hydrology.py
        noise.py
        passability.py
        predictor.py
        resources.py
        telemetry.py
        world_presets.py
        checkpoint_io.py
        agents/
            agent.py
            behavior_classifier.py
            cognitive.py
            decision.py
            drives.py
            genetics.py
            perception.py
            reproduction.py
            simulation.py
```

```

portal/                                # The public research portal
  README.md
  GWAS.md                              # Complete spec (this
document set)
  ... (see 4 for full structure)

shared/                                # NEW      canonical
contracts
  experiment_schema.json               # JSON Schema for valid
experiment exports
  types.ts                             # TypeScript types (auto-
gen from schema)

tools/                                 # NEW      CLI utilities
  export_experiment.py                 # Validates + packages
experiment as .zip
  validate_schema.py                   # Validates folder against
  experiment_schema.json
  generate_types.py                     # Generates types.ts from
experiment_schema.json

configs/                               # Unchanged
experiments/                           # Unchanged      raw local
runs
research/                              # Unchanged      papers ,
notebooks

main.py
run.py
run_test.py
run_resume.py
run_research.py
visualizer.html

```

1.3 The `shared/experiment/_schema.json` Contract

This is the most important new file in the repository. It defines exactly what a valid Genesis experiment export must contain. The simulation engine validates against it on export. The portal validates against it on upload. If they disagree, the schema is correct.

See 3 (Canonical Experiment Schema) for the full schema definition.

Part II Technology Stack

2.1 Core Framework

Layer	Choice	Rationale
Framework	Next.js 14 (App Router)	SSG for experiment pages (SEO)
Language	TypeScript (strict)	Required for complex experim
Package Manager	pnpm	Faster than npm, supports wo
Styling	Vanilla CSS (CSS Modules)	Full control over design system
Charts	Recharts	Composable React chart libra
Canvas (Replay, World Map)	Raw Canvas2D	Full per-pixel control. No abs

Layer	Choice	Rationale
Lineage Tree	\textbf{D3.js} (dendrogram only)	Use only for the hierarchical t
Schema Validation	\textbf{Zod}	Runtime type-safe validation
ZIP Parsing	\textbf{JSZip}	Client and server-side ZIP ha
CSV Parsing	\textbf{Papa Parse}	Fast, streaming CSV parsing
Markdown Rendering	\textbf{react-markdown} + \textbf{remark-gfm}	Render research documents w

2.2 Backend & Infrastructure

Layer	Choice	Rationale
Database	\textbf{Supabase} (Postgres)	Managed Postgres + Auth + Storage in one platform. No
Authentication	\textbf{Supabase Auth}	Email/password for admin. Row-level security for all tabl
File Storage	\textbf{Supabase Storage}	S3-compatible, CDN-backed, policy-based access control
Deployment	\textbf{Vercel}	Zero-config Next.js deployment, edge functions, analytics
CDN	\textbf{Cloudflare}	DNS, DDoS protection, caching in front of Vercel
Analytics	\textbf{Vercel Analytics}	Privacy-friendly, built-in to Vercel plan
OG Images	\textbf{Vercel OG} (@vercel/og)	Auto-generate per-experiment open graph images

2.3 Development Tools

Tool	Purpose
ESLint	Linting (use existing \texttt{eslintignore})
Prettier	Formatting (use existing \texttt{prettierignore})
TypeScript strict	No implicit any, strict null checks
\texttt{next/image}	Optimized image delivery for world thumbnails

Part III Canonical Experiment Schema

3.1 Required Files in Export ZIP

An experiment \texttt{.zip} is valid if and only if it contains all REQUIRED files and all files pass field validation.

File	Required	Description
\texttt{config.json}	\textbf{REQUIRED}	Full simulation config at save time
\texttt{summary.json}	\textbf{REQUIRED}	Derived metrics, distributions, leaderboards
\texttt{events.json}	\textbf{REQUIRED}	Chronological event timeline array
\texttt{population.csv}	\textbf{REQUIRED}	Tick-sampled population counts
\texttt{timeline.csv}	\textbf{REQUIRED}	Per-tick detailed stats
\texttt{births.csv}	\textbf{REQUIRED}	One row per birth event
\texttt{deaths.csv}	\textbf{REQUIRED}	One row per death event
\texttt{genes.csv}	\textbf{REQUIRED}	Per-agent gene values
\texttt{colonies.csv}	\textbf{REQUIRED}	Colony metadata
\texttt{agent_census.csv}	\textbf{REQUIRED}	Per-agent final census
\texttt{replay.json}	OPTIONAL	Full tick-by-tick replay data

File	Required	Description
\textttcheckpoint_*.json	OPTIONAL	Mid-run state checkpoints
\texttttevolution_journal.md	OPTIONAL	Auto-generated evolution journal
\textttworld.png	OPTIONAL	World map thumbnail (512px)
\textttreport.md	OPTIONAL	Auto-generated text summary report
\textttsimulation.log	OPTIONAL	Full simulation log (download only)
\textttperformance_log.csv	OPTIONAL	Per-tick perf data
\texttttcognition.csv	OPTIONAL	Per-agent cognitive stats
\texttttdisputes_history.json	OPTIONAL	Territorial dispute records
\texttttdehydration_death_traces.json	OPTIONAL	Dehydration telemetry
\texttttevolution_replay.json	OPTIONAL	Genetic history for replay slider

3.2 \textttconfig.json Full Field Specification

```

interface ExperimentConfig {
  seed:          number;      // World generation seed (
    integer)
  ticks:         number;      // Total ticks completed (
    integer)
  scarcity:      number;      // Resource multiplier [0.1,
    10.0]
  max_population: number;      // Carrying capacity (integer)
  mutation_rate: number;      // Gene mutation std dev [0.0,
    0.2]
  reproduction_enabled: boolean;
  disputes_enabled:   boolean;
  disasters_enabled:   boolean;
  healing_speed_mult:  number;
  shelter_build_speed_mult: number;
  shelter_search_dist: number;
  planner_enabled:     boolean;
  sleep_consolidation_enabled: boolean;
  checkpoint_interval: number;
  world_preset:        'island_chains' | 'arid_continent' | '
    green_continent' |
                        'boreal_highlands' | 'tropical_ring' | null;
  climate_epoch_mode: 'legacy' | 'stable' | 'slow_change' | '
    rapid_change' | 'random';
  ecology_ablation: {
    dehydration_ramp:    boolean;
    memory_fidelity:     boolean;
    water_caching:       boolean;
    deposit_utility_fix: boolean;
  };
};

```

3.3 \textttsummary.json Full Field Specification

```

interface ExperimentSummary {
  timestamp:    string;      // ISO 8601 datetime
  experiment:   string;      // Human name (from EXPERIMENT_NAME
    config var)
  seed:         number;

```

```

ticks:          number;          // Actual ticks run (may differ from
    config if stopped early)
scarcity:        number;
survivors:       string;          // "0/18" format: "<alive>/<total_born>"
avg_radius:      number;          // Average exploration radius in grid
    cells
avg_discoveries: number;
tests_passed:    boolean;
max_generation:  number;          // Highest generation number seen

derived_metrics: {
    avg_generation_interval:      number;    // Ticks per generation
    population_doubling_time:     number;    // -1 if population never
    doubled
    colony_lifespans: {            [colonyName: string]: number }; //
    Ticks
    avg_genetic_diversity:         number;    // Shannon entropy [0.0,
    1.0]
    food_efficiency:              number;
    water_efficiency:             number;
    energy_efficiency:            number;
    avg_prediction_error:         number;
    concept_formation_rate:       number;
    procedure_creation_rate:      number;
    avg_children:                 number;
    avg_grandchildren:            number;
    avg_descendants:               number;
    avg_distance_from_birthplace: number;    // Grid cells
    avg_shelter_occupancy:        number;    // [0.0, 1.0]
    avg_social_degree:            number;    // Mean relationships per
    agent
    avg_cooperation_score:         number;
    avg_conflict_score:           number;
};

distributions: {
    lifespan:      { [bucket: string]: number };
    children:      { [bucket: string]: number };
    shelter:       { [level: string]: number };
    generation:    { [gen: string]: number };
    prediction_confidence: { [range: string]: number };
    concepts:      { [range: string]: number };
};

leaderboards: {
    longest_surviving: ColonyEntry[];
    largest_size:      ColonyEntry[];
    highest_health:    ColonyEntry[];
    most_generations:  ColonyEntry[];
    cognitive_mastery: ColonyEntry[];
};
}

interface ColonyEntry {
    founder_id:    number;
    colony_id:     number;
    size:          number;

```

```

    longevity_ticks:      number;
    max_generation:       number;
    avg_health:           number;
    avg_prediction_accuracy: number;
}

```

3.4 \texttt{events.json} Event Type Definitions

```

type EventType = 'Birth' | 'Death' | 'Milestone' | 'Disaster' | '
    Dispute' |
                'ClimateEpoch' | 'Extinction';

interface BaseEvent {
    tick:      number;
    year:      number;
    day:       number;
    type:      EventType;
    description: string;
    metadata:  EventMetadata;
}

type EventMetadata =
    | BirthMetadata
    | DeathMetadata
    | MilestoneMetadata
    | DisasterMetadata
    | DisputeMetadata
    | ClimateEpochMetadata
    | ExtinctionMetadata;

interface BirthMetadata {
    child_id:      number;
    parent_a_id:   number;
    parent_b_id:   number;
    generation:    number;
    location:      [number, number]; // [y, x]
    colony_id:     number;
    colony_name:   string;
}

interface DeathMetadata {
    agent_id:      number;
    cause:         'dehydration' | 'starvation' | 'injury' | 'old_age' | '
        cold' | 'heat' | 'unknown';
    age_ticks:     number;
    location:      [number, number];
    colony_id:     number;
}

interface MilestoneMetadata {
    population?: number;
    [key: string]: unknown; // Free-form additional fields
}

interface DisasterMetadata {
    disaster_type: 'drought' | 'cold_wave' | 'heatwave' | 'famine';
    severity:      number; // [0.0, 1.0]
}

```

```

}

interface DisputeMetadata {
  agent_a_id: number;
  agent_b_id: number;
  outcome:    'retreat_a' | 'retreat_b' | 'mutual_retreat' | 'fight';
  location:   [number, number];
}

interface ClimateEpochMetadata {
  epoch_name:    string;
  temperature_delta: number;    // C offset
  rainfall_mult:  number;    // Multiplier on rainfall
}

interface ExtinctionMetadata {
  colony_id:    number;
  colony_name:  string;
  tick_founded: number;
  tick_extinct: number;
  total_members: number;
}

```

3.5 CSV Column Headers

\textbf\texttt{population.csv}

tick,total,alpha,beta,gamma,delta

\textbf\texttt{timeline.csv}

tick,year,day,alive,births_cumulative,deaths_cumulative,avg_health,
avg_hunger,avg_thirst,avg_fear,avg_generation,genetic_diversity

\textbf\texttt{births.csv}

tick,year,day,child_id,parent_a_id,parent_b_id,generation,
colony_id,colony_name,location_y,location_x

\textbf\texttt{deaths.csv}

tick,year,day,agent_id,cause,age_ticks,colony_id,
location_y,location_x

\textbf\texttt{genes.csv}

agent_id,colony_id,generation,g_metabolism,g_thermoregulation,
g_vision,g_mobility,g_memory_fidelity,g_planning,g_novelty_seeking,
g_social_proximity,g_aggression,g_resource_sharing,g_risk_sensitivity,
g_resilience,g_longevity,g_learning_rate

\textbf\texttt{agent_census.csv}

agent_id,colony_id,colony_name,generation,age_ticks,lifespan_ticks,
health_at_death,children_count,shelter_level,cause_of_death,
birth_location_y,birth_location_x,death_location_y,death_location_x,
exploration_radius,discoveries

\textbf\texttt{colonies.csv}


```
colony_id,colony_name,color_r,color_g,color_b,founder_count,
spawn_location_y,spawn_location_x,total_members,max_generation,
tick_founded,tick_extinct
```

Part IV Database Schema (Supabase Postgres)

4.1 Table: \texttt{experiments}

```
CREATE TABLE experiments (
  -- Identity
  id TEXT PRIMARY KEY,
  -- Format: 'EXP-YYYYMMDD-XXXX' (canonical) or timestamp slug
  -- Example: 'EXP-20260713-0001'

  slug TEXT UNIQUE NOT NULL,
  -- URL-safe human title: 'territorial-conflict-extreme-scarcity'

  -- Metadata written by researcher on publish
  title TEXT NOT NULL,
  abstract TEXT, -- Researcher-written
  markdown
  tags TEXT[] DEFAULT '{}',

  -- Key config fields (denormalized for fast filtering)
  seed INTEGER NOT NULL,
  ticks INTEGER NOT NULL,
  scarcity REAL NOT NULL,
  world_preset TEXT,
  climate_epoch_mode TEXT,
  max_population INTEGER,
  mutation_rate REAL,

  -- Key summary fields (denormalized for fast card rendering)
  total_agents INTEGER, -- From survivors
  denominator
  survivors_count INTEGER, -- From survivors
  numerator
  max_generation INTEGER,
  avg_genetic_diversity REAL,

  -- Full JSON blobs (for detail pages not used in list queries)
  config_json JSONB NOT NULL,
  summary_json JSONB NOT NULL,

  -- Storage
  storage_path TEXT NOT NULL, -- 'experiments/{id}/'
  prefix
  thumbnail_url TEXT, -- Public URL to world.
  png
  has_replay BOOLEAN DEFAULT FALSE, -- Whether replay.json
  is_available
  replay_size_mb REAL, -- File size for display

  -- Publishing
  is_published BOOLEAN DEFAULT FALSE,
```

```

    is_featured      BOOLEAN DEFAULT FALSE,
    published_at     TIMESTAMPTZ,
    created_at       TIMESTAMPTZ DEFAULT NOW(),
    updated_at       TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes for common filter/sort patterns
CREATE INDEX idx_experiments_published ON experiments(is_published,
    published_at DESC);
CREATE INDEX idx_experiments_scarcity  ON experiments(scarcity) WHERE
    is_published;
CREATE INDEX idx_experiments_preset    ON experiments(world_preset)
    WHERE is_published;
CREATE INDEX idx_experiments_featured  ON experiments(is_featured)
    WHERE is_published;
CREATE INDEX idx_experiments_tags      ON experiments USING GIN(tags);

```

4.2 Table: \texttt{experiment_}_events

```

CREATE TABLE experiment_events (
    id                BIGSERIAL PRIMARY KEY,
    experiment_id     TEXT NOT NULL REFERENCES experiments(id) ON DELETE
        CASCADE,
    tick              INTEGER NOT NULL,
    year              INTEGER,
    day               INTEGER,
    event_type        TEXT NOT NULL,
    description       TEXT,
    metadata          JSONB
);

CREATE INDEX idx_events_exp_id    ON experiment_events(experiment_id);
CREATE INDEX idx_events_type     ON experiment_events(experiment_id,
    event_type);
CREATE INDEX idx_events_tick     ON experiment_events(experiment_id,
    tick);

```

4.3 Table: \texttt{experiment_}_agents

```

CREATE TABLE experiment_agents (
    id                BIGSERIAL PRIMARY KEY,
    experiment_id     TEXT NOT NULL REFERENCES experiments(id) ON DELETE
        CASCADE,
    agent_id          INTEGER NOT NULL,
    colony_id         INTEGER,
    colony_name       TEXT,
    generation        INTEGER,
    age_ticks         INTEGER,
    lifespan_ticks    INTEGER,
    health_at_death   REAL,
    children_count    INTEGER DEFAULT 0,
    shelter_level     INTEGER DEFAULT 0,
    cause_of_death    TEXT,
    birth_location    JSONB,      -- {y: number, x: number}
    death_location    JSONB,      -- {y: number, x: number}
    exploration_radius REAL,

```

```

    genes          JSONB          -- {g_metabolism: 0.72, g_vision: 0.44,
    ...}
);

CREATE INDEX idx_agents_exp_id    ON experiment_agents(experiment_id);
CREATE INDEX idx_agents_colony    ON experiment_agents(experiment_id,
    colony_id);
CREATE INDEX idx_agents_gen       ON experiment_agents(experiment_id,
    generation);

```

4.4 Table: \texttt{experiment_}_population

```

CREATE TABLE experiment_population (
    id                BIGSERIAL PRIMARY KEY,
    experiment_id     TEXT NOT NULL REFERENCES experiments(id) ON DELETE
        CASCADE,
    tick              INTEGER NOT NULL,
    total              INTEGER,
    alpha              INTEGER,
    beta               INTEGER,
    gamma              INTEGER,
    delta              INTEGER
);

CREATE INDEX idx_pop_exp_tick ON experiment_population(experiment_id,
    tick);

```

4.5 Table: \texttt{research_}_documents

```

CREATE TABLE research_documents (
    id                TEXT PRIMARY KEY,
    -- e.g. 'monograph', 'architecture', 'lab-notebook', 'thesis'

    title             TEXT NOT NULL,
    document_type      TEXT NOT NULL,
    -- 'monograph' | 'architecture' | 'lab_notebook' | 'thesis' | 'paper'

    content_md         TEXT NOT NULL,          -- Full markdown content
    storage_path        TEXT,                  -- PDF path in Supabase
    Storage

    version             TEXT,                  -- e.g. 'Phase 8.4'
    is_published        BOOLEAN DEFAULT FALSE,
    published_at        TIMESTAMPTZ,
    updated_at          TIMESTAMPTZ DEFAULT NOW(),
    created_at          TIMESTAMPTZ DEFAULT NOW()
);

```

4.6 Row-Level Security (RLS) Policies

```

-- Enable RLS on all tables
ALTER TABLE experiments      ENABLE ROW LEVEL SECURITY;
ALTER TABLE experiment_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE experiment_agents ENABLE ROW LEVEL SECURITY;
ALTER TABLE experiment_population ENABLE ROW LEVEL SECURITY;

```

```

ALTER TABLE research_documents ENABLE ROW LEVEL SECURITY;

-- Public read access (published only)
CREATE POLICY "public_read_experiments"
  ON experiments FOR SELECT
  USING (is_published = TRUE);

CREATE POLICY "public_read_events"
  ON experiment_events FOR SELECT
  USING (EXISTS (
    SELECT 1 FROM experiments e
    WHERE e.id = experiment_id AND e.is_published = TRUE
  ));

-- Same pattern for agents, population, research_documents

-- Admin full access (authenticated user matching admin email)
CREATE POLICY "admin_all_experiments"
  ON experiments FOR ALL
  USING (auth.jwt() ->> 'email' = 'vinay@khosya.com');

-- Apply same admin policy to all tables

```

4.7 Supabase Storage Buckets

Bucket	Path Convention	Access
\textttexperiment-zips	\textttexperiment_id/export.zip	Private (admin write, signed URL read)
\textttthumbnails	\textttexperiment_id/world.png	Public
\textttreplays	\textttexperiment_id/replay.json	Public (large files, CDN cached)
\textttresearch-pdfs	\textttdoc_id/document.pdf	Public
\textttstaging	\textttuuid/export.zip	Private (temp, deleted after publish)

Part V REST API Specification

All endpoints under \texttt/api/v1/.

5.1 Experiments Public

```

GET /api/v1/experiments
  Query: page=1, limit=12, worldPreset, minScarcity, maxScarcity,
        tags (comma-separated), sortBy=published_at|ticks|scarcity,
        sortOrder=desc|asc, featured=true
  Returns: {
    experiments: ExperimentCard[],
    total: number,
    page: number,
    totalPages: number
  }

GET /api/v1/experiments/:id
  Returns: ExperimentDetail

```

```

GET /api/v1/experiments/:id/events
  Query: types (comma-separated), page, limit
  Returns: { events: BaseEvent[], total: number }

GET /api/v1/experiments/:id/agents
  Query: colonyId, generation, sortBy=lifespan|generation|health
  Returns: { agents: AgentRecord[] }

GET /api/v1/experiments/:id/population
  Returns: { population: PopulationEntry[] }

GET /api/v1/experiments/:id/replay-url
  Returns: { url: string, expiresAt: string, sizeMb: number }
  Note: Returns a signed URL (valid 1 hour) to replay.json in Storage

GET /api/v1/experiments/compare
  Query: a (experiment_id), b (experiment_id)
  Returns: {
    a: ExperimentDetail,
    b: ExperimentDetail,
    diff: {
      [metricKey: string]: { a: number, b: number, delta: number,
    deltaPercent: number }
    }
  }

GET /api/v1/experiments/featured
  Returns: ExperimentCard (the single featured experiment)

```

5.2 Research Public

```

GET /api/v1/research
  Returns: { documents: ResearchDocumentSummary[] }

GET /api/v1/research/:id
  Returns: ResearchDocument (with full content_md)

```

5.3 Admin Protected (requires Supabase Auth JWT)

All admin routes validate the request has a valid JWT matching the admin account.

```

POST /api/v1/admin/experiments/upload
  Body: multipart/form-data
    file: Blob (.zip)
  Returns: {
    stagingId: string,
    validation: ValidationReport,
    preview: ExperimentPreview (auto-extracted metadata)
  }

POST /api/v1/admin/experiments/:stagingId/publish
  Body: {
    title: string,
    abstract?: string, // Markdown
    tags: string[],

```

```

    isFeatured: boolean,
    experimentId?: string // Override auto-generated ID
  }
  Returns: { id: string, slug: string, url: string }

PATCH /api/v1/admin/experiments/:id
  Body: Partial<{title, abstract, tags, isFeatured, isPublished}>
  Returns: ExperimentDetail (updated)

DELETE /api/v1/admin/experiments/:id
  Returns: { success: boolean }

POST /api/v1/admin/research
  Body: {
    id:          string,
    title:       string,
    documentType: string,
    contentMd:   string,
    version?:    string
  }
  Returns: { id: string, url: string }

PATCH /api/v1/admin/research/:id
  Body: Partial<{title, contentMd, isPublished, version}>
  Returns: ResearchDocument

```

5.4 Response Type Definitions

```

interface ExperimentCard {
  id:          string;
  slug:        string;
  title:       string;
  worldPreset: string | null;
  scarcity:    number;
  ticks:       number;
  ticksHuman:  string;           // "7,970 ticks (22 yr)"
  totalAgents: number;
  survivors:   number;
  survivorsStr: string;         // "0 of 18 survived"
  maxGeneration: number;
  publishedAt: string;         // ISO 8601
  thumbnailUrl: string | null;
  isFeatured:  boolean;
  hasReplay:   boolean;
  tags:        string[];
}

interface ExperimentDetail extends ExperimentCard {
  abstract:    string | null;
  configJson:  ExperimentConfig;
  summaryJson: ExperimentSummary;
  replayUrl?:  string;         // Only if hasReplay
}

interface ValidationReport {
  valid:    boolean;
  errors:   ValidationError[];
}

```

```

    warnings: ValidationWarning[];
    preview: ExperimentPreview;
}

interface ValidationError {
    file: string; // 'summary.json'
    path: string; // '/derived_metrics/food_efficiency'
    expected: string; // 'number'
    received: string; // 'null'
    message: string; // Human-readable
}

interface ExperimentPreview {
    autoTitle: string; // From config experiment_name,
    formatted
    suggestedTags: string[]; // ['island_chains', 'extreme_scarcity', ...]
    extractedMetrics: {
        seed, ticks, scarcity, worldPreset, totalAgents,
        survivorsStr, maxGeneration, colonyNames: string[]
    }
}

```

Part VI Admin Upload & Ingestion Pipeline

6.1 Step-by-Step Pipeline

STEP 1 RECEIVE

User drops `.zip` onto upload zone.
 Client: check `file.type` = 'application/zip' and size < 500MB.
 Client: show upload progress bar.
 Upload to: Supabase Storage bucket 'staging/{uuid}/export.zip'
 On success: POST /api/v1/admin/experiments/upload with stagingId.

STEP 2 EXTRACT

Server: download `zip` from staging, extract `in-memory` (JSZip).
 List `all` files `in` the archive.
 Check `all` REQUIRED files exist.
 If missing: `return` ValidationReport with specific missing files.

STEP 3 PARSE & VALIDATE

For each required `file`:

```

config.json:
  Parse JSON. Validate against ExperimentConfig interface (Zod).
  Required: seed (integer), ticks (integer), scarcity (number > 0).

summary.json:
  Parse JSON. Validate against ExperimentSummary interface (Zod).
  Cross-check: summary.seed must equal config.seed.

```

Cross-check: summary.ticks must be $\leq$ config.ticks.

events.json:

Parse JSON. Must be an array.

Validate each element against BaseEvent (Zod discriminated union on 'type').

Warning (not error) if 0 events of a particular type (e.g. 0 disasters).

population.csv (Papa Parse streaming):

Required columns: tick, total, alpha, beta, gamma, delta.

All values must be non-negative integers.

Tick values must be monotonically increasing.

timeline.csv, births.csv, deaths.csv, genes.csv, agent_census.csv:

Required column headers checked.

No data validation (too slow for large files trusted from engine).

Cross-validation:

Every agent_id in births.csv must appear in genes.csv.

Colony names in summary.json must match those in colonies.csv (if present).

STEP 4 AUTO-EXTRACT METADATA

From validated files, build ExperimentPreview:

autoTitle: Format config.experiment_name as title case

suggestedTags: [world_preset, 'scarcity_extreme'|'scarcity_high'|'scarcity_normal',

climate_epoch_mode, 'fights_enabled' if

disputes_enabled, ...]

totalAgents: Parse denominator from summary.survivors

survivorsStr: Format "X of Y survived"

colonyNames: From colonies.csv or default ['Alpha','Beta','Gamma','Delta']

STEP 5 DATABASE INSERT (draft)

Generate experiment_id: 'EXP-{YYYYMMDD}-{4-char-hash}'

Insert experiments row: is_published = FALSE

Batch-insert experiment_events rows (chunked at 1000 rows)

Batch-insert experiment_agents rows

Batch-insert experiment_population rows (downsampled to max 5000 rows)

STEP 6 RETURN PREVIEW

Return: { stagingId, validation, preview }

Client renders: full preview page (same layout as published page)

Client shows: metadata form (title, abstract, tags, featured toggle)

STEP 7 PUBLISH

POST /api/v1/admin/experiments/:stagingId/publish with form data.


```

Server:
  UPDATE experiments SET is_published=TRUE, published_at=NOW(), title,
    abstract, tags, isFeatured
  Copy zip from staging/ to experiments/{id}/export.zip
  Copy world.png (if present) to thumbnails/{id}/world.png
  Copy replay.json (if present) to replays/{id}/replay.json
  Update experiments.has_replay, thumbnail_url
  Invalidate Vercel cache: revalidatePath('/civilizations'),
    revalidatePath('/')
  Return: { id, slug, url }
Client: toast "Published"      View live      "

```

6.2 Downsampling Strategy for \texttt{experiment}_population

For experiments > 100,000 ticks, the raw population.csv may have >20,000 rows. Downsample to 5000 rows using:

```

# Adaptive downsampling: keep all rows for tick < 1000 (early dynamics)
,
# then sample at regular intervals for the remainder.
sample_every = max(1, len(rows) // 5000)
kept = [r for i, r in enumerate(rows) if r.tick < 1000 or i %
    sample_every == 0]

```

6.3 Replay File Strategy

Replay files can be very large (a 1M-tick run would be enormous). Policy:

- If \texttt{replay.json} is < 50MB: upload as-is, served directly
- If \texttt{replay.json} is 50-200MB: upload compressed (gzip), decompress client-side
- If \texttt{replay.json} is > 200MB: only upload first 50,000 ticks. Mark \texttt{replay}_truncated: true in experiments table.
- If \texttt{replay.json} is absent: \texttt{has_replay} = false. Replay tab shows "Replay not available for this civilization."

Part VII Folder Structure (Portal)

```

portal/

  app/                                     # Next.js App Router
    page.tsx                             # / Home
    layout.tsx                           # Root layout

    civilizations/
      page.tsx                           # /civilizations
Archive
      compare/
        page.tsx                         # /
civilizations/compare
[id]/

```

```

page.tsx # /civilizations
/[id] Overview layout.tsx # Experiment
layout (sidebar, tabs) observatory/page.tsx # Charts &
analytics replay/page.tsx # Canvas replay
chronicle/page.tsx # Events timeline
world/page.tsx # World map
lineage/page.tsx # Agent roster +
tree genome/page.tsx # Gene
distributions conditions/page.tsx # Config viewer

research/
  page.tsx # Research hub
  monograph/page.tsx
  architecture/page.tsx
  field-notes/page.tsx
  thesis/page.tsx

timeline/page.tsx # Living Timeline
origin/page.tsx # About Genesis
collaborate/page.tsx # Contact/
Collaborate docs/
  page.tsx
  schema/page.tsx
  config/page.tsx
  engine/page.tsx

publications/page.tsx # Reserved
empty state datasets/page.tsx # Reserved
empty state api/page.tsx # Reserved
empty state community/page.tsx # Reserved
empty state citations/page.tsx # How to cite
Genesis version-history/page.tsx # Engine changelog

admin/
  layout.tsx # Auth guard
  page.tsx # Command dashboard
  upload/page.tsx
  civilizations/page.tsx
  research/page.tsx

api/ # Next.js API routes
  v1/
    experiments/
      route.ts # GET /api/v1/
experiments compare/route.ts
[id]/

```

```

        route.ts
        events/route.ts
        agents/route.ts
        population/route.ts
        replay-url/route.ts
    research/
        route.ts
        [id]/route.ts
    admin/
        experiments/
            upload/route.ts
            [stagingId]/publish/route.ts
        research/route.ts

components/
    layout/
    civilization/
    charts/
    replay/
    world/
    agent/
    timeline/
    admin/
    ui/

lib/
    supabase.ts
    supabase-server.ts                                # Server-side
client (service role)
    api.ts
    schema-validator.ts                                # Zod schemas for
all experiment files
    experiment-parser.ts                                # ZIP extraction +
CSV/JSON parsing
    experiment-ingestion.ts                            # Full upload
pipeline logic
    replay-renderer.ts                                # Canvas2D rendering

hooks/
    useExperiment.ts
    useReplay.ts
    useWorldMap.ts

types/
    experiment.ts
    agent.ts
    event.ts
    genome.ts

styles/
    globals.css                                        # Design system
tokens (from GDS)
    typography.css
    animations.css

public/
    fonts/
    icons/

```

```

next.config.js
tsconfig.json
package.json
.env.local.example

```

Part VIII Environment Variables

```

# .env.local.example

# Supabase (required)
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key      # Safe to expose
RLS enforced
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key  # NEVER expose to
client

# Admin configuration
ADMIN_EMAIL=vinay@khosya.com                    # The only admin
account

# Optional Analytics
NEXT_PUBLIC_VERCEL_ANALYTICS_ID=your-id

```

Part IX Deployment Architecture

9.1 Infrastructure Diagram

```

User Browser
    DNS lookup      Cloudflare (Proxy enabled)
Cloudflare
    CDN cache hit   return cached response
    cache miss

Vercel Edge Network
    Static assets (/fonts, /icons, /public)      CDN served
    SSG pages (/civilizations/[id])              cached at build or ISR
    API routes (/api/v1/*)                       Vercel serverless functions
    Admin routes (/admin/*)                      SSR, no caching

    Supabase (ap-south-1 or us-east-1)
        Postgres      experiments, events, agents, population,
research
        Auth          admin session management
        Storage       zips, thumbnails, replays, PDFs

```

9.2 Caching Strategy (ISR)

```

// /app/civilizations/[id]/page.tsx
export const revalidate = 3600;    // Revalidate once per hour

```

```
// /app/civilizations/page.tsx
export const revalidate = 300;      // 5 minutes      new experiments
    show up quickly

// /app/research/[doc]/page.tsx
export const revalidate = 86400;    // 24 hours

// /app/page.tsx (homepage)
export const revalidate = 300;      // 5 minutes      featured experiment
    updates
```

On publish, the admin route calls:

```
import { revalidatePath } from 'next/cache';
revalidatePath('/civilizations');
revalidatePath('/');
revalidatePath(`/civilizations/${newId}`);
```

9.3 Build Configuration

```
// next.config.js
const config = {
  images: {
    remotePatterns: [{
      protocol: 'https',
      hostname: '*.supabase.co',    // Allow Supabase Storage
      thumbnails
    }]
  },
  experimental: {
    serverComponentsExternalPackages: ['jszip', 'papaparse']
  }
};
```

9.4 Domain Setup

```
Primary: genesis.vinaykhosya.com
    Cloudflare DNS (proxied)
    Vercel project (custom domain configured)

Admin: genesis.vinaykhosya.com/admin
    Same deployment, auth-guarded at middleware level
```

Part X Security

10.1 Admin Protection

Every `\texttt/admin/*` route is protected by Supabase Auth middleware:

```
// middleware.ts
import { createMiddlewareClient } from '@supabase/auth-helpers-nextjs'

export async function middleware(req: NextRequest) {
  const res = NextResponse.next();
```

```
const supabase = createMiddlewareClient({ req, res });
const { data: { session } } = await supabase.auth.getSession();

if (req.nextUrl.pathname.startsWith('/admin')) {
  if (!session || session.user.email !== process.env.ADMIN_EMAIL) {
    return NextResponse.redirect(new URL('/login', req.url));
  }
}
return res;
}
```

10.2 Upload Security

- ZIP files are validated server-side before any database write
- Maximum file size: 500MB (enforced at Supabase Storage upload policy)
- Only \texttt{.zip} MIME type accepted
- Staging files auto-deleted after 24 hours if not published (Supabase lifecycle rule)

10.3 API Rate Limiting

- Public endpoints: 100 requests/minute per IP (Vercel Edge Config)
- Admin endpoints: 20 requests/minute per session
- Upload endpoint: 5 requests/minute per session

\textit{End of GES v1.0}

\textbf{Document produced by Antigravity AI, July 2026 \textbf{Companion: GPS GDS GCR}}